

OTNT

One-Time Network Tunnel

A Self-Destructing WireGuard VPN — End-to-End Project Study

Final Year Project — Cybersecurity | Prepared for jury review

Document version: complete, empirically verified system + academic evaluation

Why we built this

Most VPN access, once granted, quietly stays granted. A contractor gets a VPN profile for a one-day job and it is still valid a year later because nobody remembered to revoke it. That gap between “I needed access for one task” and “this access technically never expired” is the actual attack surface in a lot of real breaches — not a broken firewall, just a forgotten door.

OTNT asks a simple question: what if a VPN tunnel behaved like a prepaid mobile recharge pack instead of a permanent account? A prepaid pack is valid for N days **or** until the data runs out — whichever happens first — and then it just stops, automatically, with nobody needing to remember to cancel it.

OTNT applies exactly that idea to a real, kernel-level WireGuard tunnel: every tunnel is created with a time limit, a data limit, or both, and it tears itself down the instant either limit is hit. No standing access, no manual revocation step to forget.

Everything in this document describes a system that is actually built, actually running real Linux WireGuard interfaces (not a simulation), and actually measured with real experiments — not a paper design.

1. Objectives

The project was scoped into four concrete objectives, built in order:

#	Objective	In one line
1	Core self-destructing tunnel	Build a real (not simulated) WireGuard tunnel that automatically destroys itself on a time limit, a data limit, or both — whichever comes first.
2	Durable state (persistence)	Make tunnel records survive a backend crash or restart, so a live tunnel is never lost track of or double-managed.
3	Deployment hardening	Make the system safe to actually run in the real world: configurable CORS, per-endpoint rate limits, durable Redis storage, and an operator recovery tool.
4	Empirical validation	Prove the core claim and characterize real-world precision with real experiments, compared honestly against relevant baselines where they exist.

IN PLAIN TERMS

Objective 1 is the actual product (the recharge-pack tunnel). Objective 2 is making sure the phone company's billing system doesn't forget your pack exists if their server restarts. Objective 3 is making sure the system can survive real customers hammering it, not just a demo. Objective 4 is proving all of this with real receipts, not a claim on a slide.

2. The End-to-End Flow, in One Pass

One sentence version: a client asks for a tunnel → the server builds a real WireGuard interface and hands back an encrypted configuration → the client connects → two independent timers (time, data) watch the tunnel → whichever limit is hit first tears the tunnel down completely, automatically, and writes an audit record. Section 3 walks each of these seven steps in detail.

Step	Stage	Trigger
1	Handshake	Client opens the app, generates its own keys, asks the server for a tunnel slot
2	Real tunnel creation	Server brings up a genuine Linux WireGuard kernel interface
3	Encrypted config delivery	Client fetches and decrypts its connection profile, in-browser
4	Live monitoring	Client watches a live countdown / data-used meter
5	Dual-condition self-destruct	Time OR data cap hit → tunnel tears itself down
6	Manual delete / crash recovery	User can end early; server restart never loses track of live tunnels
7	Concurrency	Many tunnels run side-by-side without colliding

This is the exact sequence exercised by every real test in Section 4.

3. Step-by-Step, With the Reasoning Behind Each Choice

STEP 1

Handshake — asking for a tunnel

What happens: The browser generates a WireGuard keypair **and** a separate ECDH keypair locally, and sends only the ECDH public key to POST /api/handshake/init. The server generates its own ECDH keypair, derives a shared secret, and allocates a private IP pair for this tunnel.

Tech stack: React frontend, TweetNaCl (X25519) key generation in-browser, Node.js/Express backend.

Why we did it this way: The client's private keys must never leave the device — the server should never be able to impersonate a client even if it were compromised. Exchanging only public keys and deriving a shared secret independently on each side (ECDH) achieves this without ever transmitting a secret.

IN PLAIN TERMS

Like two people agreeing on a secret handshake by each doing their own private math on a number the other one shared publicly — anyone watching sees the public numbers, but only the two of them can compute the actual shared secret.

STEP 2

Real tunnel creation on the server

What happens: POST /api/tunnel/create allocates a fresh interface name (otnt0, otnt1, ...), a free UDP port, calls the real wg-quick up command, and registers the new interface in Redis as OTNT-owned **before** bringing it up.

Tech stack: Node.js shelling out to wg/wg-quick/ip, Redis ownership registry, a serializing allocation lock so two simultaneous requests can never grab the same port/IP.

Why we did it this way: A simulated tunnel wouldn't actually protect real traffic, so this had to be a genuine Linux kernel network interface. The ownership registry exists so the system can never mistake someone else's unrelated network interface for one of its own and accidentally delete it.

IN PLAIN TERMS

Like a hotel issuing a brand-new physical lock and key for every guest room, and writing the room number in the front-desk ledger the instant the lock is fitted — never after — so housekeeping can never mix up which room belongs to which guest.

STEP 3

Encrypted configuration delivery

What happens: The client requests its config from POST /api/tunnel/:id/client-config using a fresh, one-time public key. The server encrypts the WireGuard profile with AES-256-GCM (keyed from a freshly derived ECDH shared secret) and sends back ciphertext. The browser decrypts it locally using WebCrypto and splices in its own private key, which the server has never seen.

Tech stack: AES-256-GCM, ECDH, WebCrypto API (browser-native, not a JS crypto library).

Why we did it this way: Even though the call already happens over the network, encrypting the payload itself means the config can never be read in transit even by something sitting between the browser and

the server. And the server never holding the client's private key means a server compromise still can't reveal it.

IN PLAIN TERMS

Like receiving a locked box in the post that only opens with a key you already have at home — even if someone intercepts the box in transit, it's useless to them without that key.

STEP 4

Live monitoring in the browser

What happens: The frontend polls GET /api/tunnel/status/:id once a second and shows a live countdown timer and a data-used progress bar.

Tech stack: React setInterval polling, Axios, Material UI progress components.

Why we did it this way: A temporary-access system should never leave the user guessing when their access ends — visibility is part of the trust model, not a cosmetic feature.

IN PLAIN TERMS

Like watching a parking meter count down while you're in the spot, instead of finding a ticket on your windscreen with no warning.

STEP 5

Dual-condition self-destruct

What happens: Two independent timers run per tunnel: one checks the clock every second, one checks bytes transferred every 250 milliseconds. The instant either condition is met, a terminating flag is set synchronously so only one of them can ever actually act, and the tunnel is torn down: real interface deleted, IP/port released, Redis record deleted, one audit line written.

Tech stack: Two setInterval monitors, a race-safe termination flag, a single shared teardown function every trigger routes through.

Why we did it this way: This is the project's actual claim — access should end by design, the moment it's no longer needed, without a human having to remember to revoke it. The race-safe flag exists because two timers checking independently could otherwise both try to tear the same tunnel down at once.

IN PLAIN TERMS

*Exactly a prepaid recharge pack: valid for N days **or** until the data runs out, whichever happens first — then it just stops. Nobody calls the phone company to cancel it.*

STEP 6

Manual delete and crash recovery

What happens: A user can end a tunnel early via POST /api/tunnel/delete. Separately, if the backend process itself crashes or restarts, it reloads every live tunnel record from Redis on startup and restarts its timers from where they left off, before doing any cleanup of stale interfaces.

Tech stack: Redis with AOF (append-only file) durability, a strict startup order (connectRedis → rehydrateTunnels → cleanupStartup).

Why we did it this way: Real servers crash and restart. A temporary-access system that forgets which tunnels were live after a restart could either leave a real tunnel open forever (a security hole) or delete a real, still-active tunnel it shouldn't have touched.

IN PLAIN TERMS

Like a hotel's electronic key-card system remembering exactly which rooms are occupied even after a power cut, instead of unlocking every door for everyone once the power comes back.

STEP 7

Concurrency — many tunnels at once

What happens: Every tunnel gets its own interface name, IP pair, and UDP port from a locked allocator, so two people creating tunnels at the same moment can never be handed the same resource.

Tech stack: A dedicated serializing lock around the allocator functions, an ownership registry backed by Redis sets.

Why we did it this way: An earlier version of this system deleted every existing tunnel every time a new one was created — creating tunnel #2 silently destroyed tunnel #1. This was found, root-caused, and fixed so many independent tunnels can run side by side safely.

IN PLAIN TERMS

Like every hotel guest getting their own separate room and key card — checking in a new guest never evicts the guest already in room 4.

4. Testing — What Was Actually Checked, and Why

4.1 Automated unit tests (Jest)

53 automated tests across 5 files run against the real production code (not a reimplementaion of it), covering the pieces that are cheapest to get subtly wrong.

Test file	What it checks
allocators.test.js	IP/port/interface allocation is unique, releases correctly, and stays collision-free even under genuinely simultaneous requests.
cryptoUtils.test.js	The ECDH + AES-GCM round trip actually encrypts and decrypts correctly, and key validation rejects malformed input.
expiryTimer.test.js	The real dual-condition timer logic fires at the right moment, not early, not late, and exactly once even when both conditions trip together.
ownership.test.js	The system only ever touches interfaces it created itself — it correctly refuses to delete an unrelated interface even with a matching name.
transferBytes.test.js	Data-usage readings fail loudly instead of silently reporting zero, which would otherwise hide real usage from the cap check.

IN PLAIN TERMS

Like a pre-flight checklist a pilot runs before every single takeoff — dozens of small, individually boring checks (does this lock actually lock? does this gauge actually read correctly?) that must all pass before anyone trusts the whole aircraft.

4.2 Real WireGuard lifecycle tests (not simulated)

Unit tests alone can't prove a tunnel actually protects real traffic or actually expires on schedule — they test logic in isolation. So the system was also tested end-to-end using real Linux network namespaces and veth pairs to create a genuine second network, with a real WireGuard client generating real encrypted traffic through the real backend — the same technique used throughout every experiment in Section 5.

- Both self-destruct paths (time limit, data limit) verified against a real kernel interface, not a mock.
- Crash-recovery verified: kill the backend mid-tunnel, restart it, confirm the tunnel is rediscovered and its timers resume correctly.
- A real 2-tunnel concurrency test: tunnel A mid-transfer while tunnel B is created — 7 separate assertions, all passed, proving simultaneous tunnels never interfere with each other.

IN PLAIN TERMS

Like doing an actual fire drill with real people walking out of a real building, instead of just checking that the fire-alarm wiring diagram looks correct on paper.

4.3 What the tests found

Testing wasn't just a formality — it surfaced two real, previously-unnoticed bugs, both fixed and re-verified live: the tunnel-endpoint display showed the wrong port number for any tunnel other than the

first, and three live status fields (interface name, server IP, client IP) never reached the dashboard header despite being available from the very API call already polling every second.

5. How This Was Benchmarked Against Baselines

Three different kinds of comparison were used, and each is labeled honestly as what it actually is — an internal comparison, a same-system baseline, or a literature reference point that is explicitly **not** claimed to be equivalent.

Comparison	Type	Result	Verdict
Dual-condition vs. time-only vs. no-limit tunnel	Internal (self vs. self)	Mean exposure window: 0.014s vs. 5.37s vs. 5.80s	~380× reduction — the project's core claim, tested directly
OTNT-provisioned tunnel vs. vanilla WireGuard	Same-system baseline	1249.5 Mbps vs. 1230.6 Mbps throughput	Statistically identical — zero measurable cost from OTNT's control layer
OTNT provisioning latency vs. Haga et al. 2020	Literature reference (not equivalent)	OTNT: 666ms, own number, no head-to-head claim made	Different domain (5G-slice orchestration) — reported side-by-side only, never compared
OTNT teardown latency vs. Chang & Xu 2026	Literature reference (not equivalent)	OTNT: ~220–240ms vs. their <3ms	Different mechanism (real kernel teardown + DB delete vs. in-memory signal) — gap explained, not hidden

IN PLAIN TERMS

The strongest, fairest comparison here is against ourselves — the same tunnel, with and without the self-destruct behaviour turned on — the same way you'd test whether a seatbelt works by comparing the same car with and without it, not by comparing it to an unrelated car from a different manufacturer.

No comparison against OpenVPN was run in this project — that comparison already exists in the cited literature and was deliberately not repeated.

6. Repository Guide — What Each File Does

Backend (Node.js / Express)

File	What it does
index.js	The Express API server. Owns every route and the dual-condition expiry/data-cap timers.
wireguard.js	All real wg/wg-quick/ip commands; the ownership registry, allocators, and the concurrency lock.
redisClient.js	Redis/Valkey connection and tunnel-record read/write — the durable source of truth.
cryptoUtils.js	Pure ECDH + AES-GCM helper functions, kept side-effect-free so tests can exercise real crypto directly.
auditLogger.js	Writes a timestamped audit line to console and a log file for every tunnel lifecycle event.
scripts/reap.js	Manual operator tool: lists, then optionally deletes, orphaned interfaces the automatic cleanup deliberately never touches on its own.
__tests__ / (5 files)	53 Jest tests covering allocators, crypto, ownership, expiry timing, and data-transfer reads — see Section 4.1.

Frontend (React / Vite)

File	What it does
App.jsx	Top-level page layout, tab navigation, and the state that ties every tunnel-lifecycle component together.
api/axios.js	A thin wrapper around the four backend endpoints the UI calls.
utils/wireguardKeys.js	Generates the client's real WireGuard keypair in the browser; the private key never leaves the device.
utils/cryptoClient.js	Client-side ECDH + AES-GCM decrypt helpers, built on the browser's native WebCrypto API.
components/TunnelForm.jsx	The tunnel-creation form; drives the handshake and create API calls.
components/TunnelStatus.jsx	Polls tunnel status every second; shows the live countdown and data-used meter, handles teardown.
components/DownloadEncryptedConfig.jsx	Fetches and decrypts the client's WireGuard profile; offers a file download and a QR code.
components/SecureSessionCard.jsx	Displays the exchanged public keys for the currently active session.
components/TunnelLifecycle.jsx	A visual pipeline view of handshake → create → decrypt → connect → self-destruct.
components/HowItWorksPanel.jsx	A static explainer panel describing the architecture, for demo viewers.
theme.js	Shared colour/typography tokens and Material UI style overrides.
main.jsx / vite.config.js	React app entry point and the Vite dev-server/build configuration.

Test Results — Consolidated

Test	Result
Jest unit suite	53 / 53 passing, 5 suites
Real 2-tunnel concurrency test	7 / 7 assertions passed on real kernel interfaces
Exposure window (dual-condition vs. persistent)	0.014s vs. 5.80s — ~380× reduction, n=20/condition
Tunnel creation latency	~340–348ms across all conditions, real end-to-end (own measurement)
Throughput vs. vanilla WireGuard	1249.5 Mbps vs. 1230.6 Mbps — statistically indistinguishable, n=10/condition
Time-expiry firing precision	Mean delta 627.5ms, flat across 5–20s targets (r=0.187), n=30
Data-cap overshoot vs. throughput	Strong linear correlation, r=0.957, n=20
Teardown-to-audit-log latency	Mean 232.1ms, n=50 combined

All figures are from real trials against the running backend and real WireGuard kernel interfaces — network-namespace/veth-pair emulation on a single machine, not simulated data and not a multi-hop network. Full raw data and statistics live in docs/experiments/ alongside each written analysis.

End of document.